

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE1104

Proyecto #1

Integrantes:

Melany Montoya Matamoros - 2025125964

Kristel Valverde Gómez – 2025143158

Profesor:

Luis Alonso Barboza Artavia

I semestre 2025

I. Introducción

El presente proyecto tiene como objetivo la construcción y programación de una máquina basada en la máquina de Tuki: "*Cats and Mice*", empleando botones con luces para crear patrones que el usuario deberá seguir. Para su elaboración se utilizará una *Raspberry Pi Pico W* como unidad de procesamiento, acompañada de código programado en *MicroPython* para gestionar la parte lógica del sistema, sonido y recopilación de datos de respuesta.

Mediante esta creación, se busca desarrollar una herramienta funcional y eficiente que pueda ser utilizada para entretenimiento y diversión, al mismo tiempo que se favorece el aprendizaje y aplicación de conocimientos en programación y electrónica. Asimismo, se busca potenciar habilidades como el trabajo en equipo y la resolución de problemas.

II. Marco Teórico

En este apartado se explicarán las bibliotecas utilizadas para la elaboración del código que gestiona la lógica de la máquina, a continuación, se enumeran y explican cada una de ellas:

- Machine: La biblioteca *machine* en *MicroPython* permite la interacción directa con el hardware de la *Raspberry Pi Pico W*, pues proporciona funciones y clases que permiten el control de periféricos como los pines GPIO. Para la elaboración del código se usaron las clases *Pin* y *PWM*, las cuales permiten crear objetos que representan los pines GPIO y señales PWM. Además, se utilizaron métodos como *.value()* para leer y establecer el estado de los pines GPIO, facilitando el control de los dispositivos conectados a la placa.
- Utime: La biblioteca *utime* en *MicroPython* permite la gestión del tiempo y las pausas en la ejecución de programas. Permite medir intervalos de tiempo, realizar retardos, entre otras funciones más. En el código del proyecto se utilizaron funciones como *.sleep()*, *.ticks_ms()*, *.ticks_diff()*, las cuales permiten pausar y obtener el tiempo del sistema respectivamente.
- Random: La biblioteca *random* en *MicroPython* (y en otros lenguajes de programación) permite la generación de números aleatorios. En el código del proyecto se utilizaron las funciones *random.randint()* y *random.sample()*, las cuales permiten devolver números dentro de un intervalo definido y obtener elementos aleatorios dentro de un bucle respectivamente.

III. Desarrollo

→ Inicialización de entradas y salidas.

Se configuran los pines GPIO que se utilizan para los *LED*'s, botones, *switch*, *buzzer* y el *display* de 7 segmentos. Esto permite leer y controlar lo que pasa con el dispositivo físico que es lo que permite la interacción entre lo que pasa en el código y la parte física que recibe el usuario.

```
9 # LEDs
10 led0 = Pin(18, Pin.OUT)
11 led1 = Pin(19, Pin.OUT)
12 led2 = Pin(20, Pin.OUT)
13 led3 = Pin(21, Pin.OUT)
14 leds = [led0, led1, led2, led3]
15
16 # Botones
17 btn0 = Pin(2, Pin.IN, Pin.PULL_DOWN)
18 btn1 = Pin(3, Pin.IN, Pin.PULL_DOWN)
19 btn2 = Pin(4, Pin.IN, Pin.PULL_DOWN)
20 btn3 = Pin(5, Pin.IN, Pin.PULL_DOWN)
21 botones = [btn0, btn1, btn2, btn3]
22
23 # Switches (pull-up: active cuando value() == 0)
24 switch = Pin(16, Pin.IN, Pin.PULL_UP) # Modo principiante si activo
25 switch2 = Pin(17, Pin.IN, Pin.PULL_UP) # Modo avanzado si activo
26
27 # Buzzer
28 buzzer = PWM(Pin(22))
29
30 # Display 7 segmentos (cátodo común)
31 segA = Pin(9, Pin.OUT)
32 segB = Pin(10, Pin.OUT)
33 segC = Pin(11, Pin.OUT)
34 segD = Pin(12, Pin.OUT)
35 segE = Pin(13, Pin.OUT)
36 segF = Pin(14, Pin.OUT)
37 segG = Pin(15, Pin.OUT)
38 segmentos = [segA, segB, segC, segD, segE, segF, segG]
```

→ Modo piano (modo_piano).

Es un minijuego musical que se encuentra separado del juego principal. El jugador accede si mantiene presionado un botón especial (botón 3). Según la posición en la que se encuentre el *switch*, será la melodía que se podrá tocar. En cada turno se enciende un *LED*, y el jugador presiona el botón correspondiente para tocar la nota asignada.

```
notas_piano = [262, 294, 330, 349] # Do, Re, Mi, Fa (frecuencias en Hz)

def modo_piano():
    print("Entrando al modo piano")
    mostrar_hex_en_display(10) # Letra A
    sleep(1)
    while btn3.value(): pass # Esperar a que suelte btn3

    botones_tocar = [btn2, btn3]
    leds_tocar = [leds[2], leds[3]]

    while True:
        melodia_sel = 0 if not switch.value() else 1 if not switch2.value() else 2
        melodia = melodias[melodia_sel]

        for idx, (frecuencia, duracion) in enumerate(melodia):
            i_led = idx % len(leds_tocar)
            leds_tocar[i_led].on()
            presionado_correctamente = False

            while not presionado_correctamente:
                # Salir menuas con btn1
                if btn1.value():
                    while btn1.value(): pass
                    for led in leds:
                        led.off()
                    mostrar_hex_en_display(0)
                    print("Saliendo del modo piano...")
                    return

            # Reiniciar con btn0
            if btn0.value():
                while btn0.value(): pass
                for led in leds:
                    led.off()
                print("Reiniciando melodía...")
                mostrar_hex_en_display(10)
                sleep(0.5)
                break

            for j, btn in enumerate(botones_tocar):
                if btn.value():
                    while btn.value(): pass
                    leds_tocar[i_led].off()
                    if j == 1:
                        sonido(frecuencia, duracion)
                        presionado_correctamente = True
                    else:
                        sonido_error()
                        break

            sleep(0.01)

        sleep(0.2)

    print("Melodía completada. Puedes reiniciar o cambiar con el switch.")
    mostrar_hex_en_display(11)
    sleep(1)
```

→ Funciones auxiliares.

Se incluyen una serie de funciones auxiliares que se encargan de funciones muy específicas dentro de la mecánica del juego.

→ *mostrar_hex_en_display(valor)*: Muestra el puntaje del jugador, en sistema numérico hexadecimal en el display de 7 segmentos.

→ *sonido(freq, duracion)*: reproduce las notas musicales según lo establecido cuando se declaran las notas o melodías.

→ *animacion_inicio, animacion_cambio_modos*: crea una animación tanto visual como sonora antes de que inicie el juego, cambio de modo y en victorial al obtener el máximo puntaje posible en el juego (16 puntos en sistema decimal)

```
64 def mostrar_hex_en_display(valor):
65     valor = max(0, min(15, valor))
66     for i in range(7):
67         segmentos[i].value(segmentos_hex[valor][i])
68
69 def sonido(freq, duracion):
70     buzzer.freq(freq)
71     buzzer.duty_u16(32768)
72     sleep(duracion)
73     buzzer.duty_u16(0)
74
```

```
78 def animacion_inicio():
79     print("[Animacion inicio]")
80     for _ in range(3):
81         for led in leds: led.on()
82         sleep(0.2)
83         for led in leds: led.off()
84         sleep(0.2)
85     for led in leds: led.on()
86     sleep(1)
87     for led in leds: led.off()
88
89 def animacion_cambio_modos():
90     print("[Animacion cambio modos]")
91     for _ in range(2):
92         for led in leds: led.on()
93         sleep(0.1)
94         for led in leds: led.off()
95         sleep(0.1)
96     for f in [880, 1100, 1320]:
97         sonido(f, 0.1)
98
```

→ **Turno o ronda de juego (turno_reflejo(modos)).**

Esta función representa un turno o ronda dentro del juego principal. Como ya se mencionó la mecánica del juego es que se enciende un *LED* aleatorio y el jugador debe presionar el botón correspondiente antes de que se acabe el tiempo según el modo que este jugando (principiante, avanzado, leyenda). Si acierta el *buzzer* emite un sonido anteriormente definido como *sonido_acierto* y retorna *True*; lo mismo si presiona el botón incorrecto o acaba el tiempo en *sonido_error* y retorna *False*.

```
99 def turno_reflejo(modos):
100     tiempo_limite = 1000 if modos == "principiante" else 500 if modos == "avanzado" else 2000
101     led_aleatorio = random.randint(0, 3)
102     print(f"[Turno reflejo] modos: {modos}, led: {led_aleatorio}, tiempo limite: {tiempo_limite}ms")
103
104     leds[led_aleatorio].on()
105     inicio = ticks_ms()
106
107     while ticks_diff(ticks_ms(), inicio) < tiempo_limite:
108         for i, btn in enumerate(botones):
109             if btn.value():
110                 while btn.value(): pass
111                 leds[led_aleatorio].off()
112                 if i == led_aleatorio:
113                     print("[Turno reflejo] Acertaste")
114                     sonido_acierto()
115                     return True
116                 else:
117                     print(f"[Turno reflejo] Fallaste, botón presionado: {i}")
118                     sonido_error()
119                     return False
120         sleep(0.01)
121
122     # Tiempo agotado
123     print("[Turno reflejo] Tiempo agotado")
124     leds[led_aleatorio].off()
125     sonido_error()
126     return False
```

→ **Modo juego principal (jugar).**

Controla la lógica principal del juego:

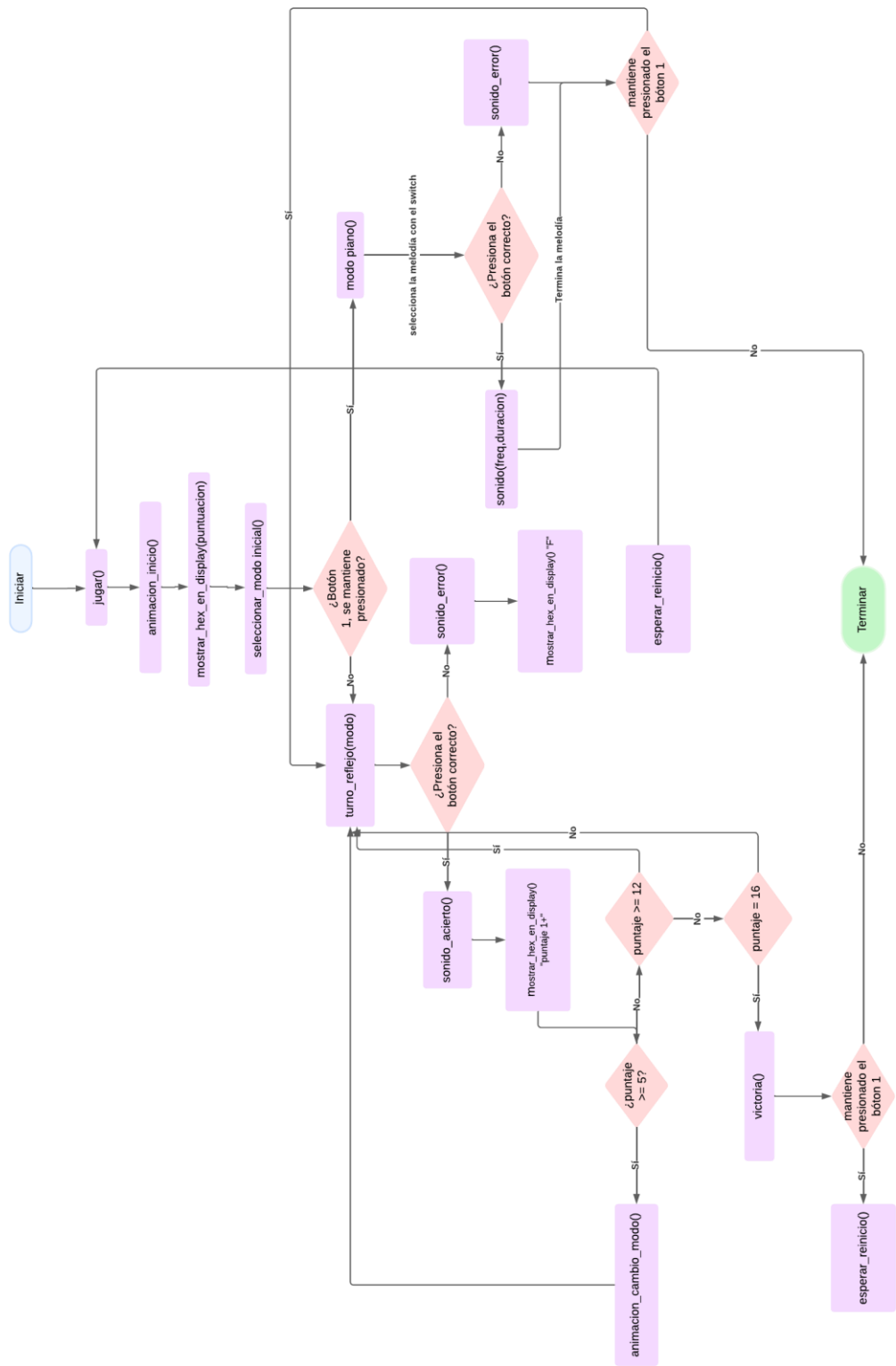
- Antes de iniciar el juego se puede seleccionar el modo en el cual se desea jugar (principiante, avanzado, leyenda).
- El juego avanza hasta que el jugador pierda u obtenga el mayor puntaje
- Se lleva el conteo de los puntos y estos se muestran en el display en sistema hexadecimal.
- Cuando se juega en modo principiante automáticamente se avanza al nivel avanzado y leyenda cuando se obtiene un puntaje específico.
- Al fallar se muestra una “F” en el display que significa “Fail” y se espera que se presione el botón 1 para reiniciar el juego.

```

226 def jugar():
227     while True:
228         puntuacion = 0
229         modo = seleccionar_modo_inicial()
230         animacion_inicio()
231         mostrar_hex_en_display(puntuacion)
232
233         # Antes de empezar, chequear si se quiere entrar al modo piano
234         print("[Esperando inicio o modo piano (btn1)]")
235         inicio_espera = ticks_ms()
236         while ticks_diff(ticks_ms(), inicio_espera) < 3000:
237             if btn1.value():
238                 modo_piano()
239                 print("[Volviendo al juego principal]")
240                 animacion_inicio()
241                 mostrar_hex_en_display(puntuacion)
242                 inicio_espera = ticks_ms()
243                 sleep(0.01)
244
245         while True:
246             # Permitir entrar a modo piano antes de perder o reiniciar
247             if btn1.value():
248                 modo_piano()
249                 animacion_inicio()
250                 mostrar_hex_en_display(puntuacion)
251
252             if modo == "principiante" and puntuacion >= 5:
253                 modo = "avanzado"
254                 animacion_cambio_modo()
255             elif modo == "avanzado" and puntuacion >= 12:
256                 modo = "leyenda"
257                 animacion_cambio_modo()
258             elif modo == "leyenda" and puntuacion >= 16:
259                 victoria()
260                 break
261
262             if turno_reflejo(modo):
263                 puntuacion += 1
264                 mostrar_hex_en_display(puntuacion)
265             else:
266                 break
267
268         esperar_reinicio()
269
270 jugar()

```

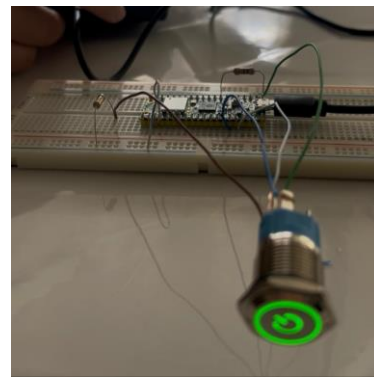
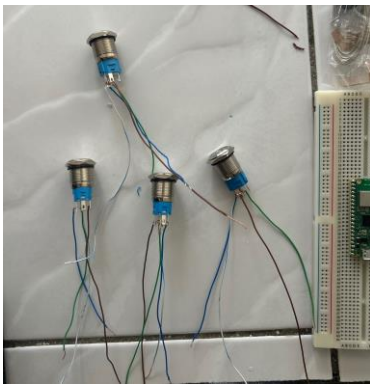
→ Diagrama de flujo



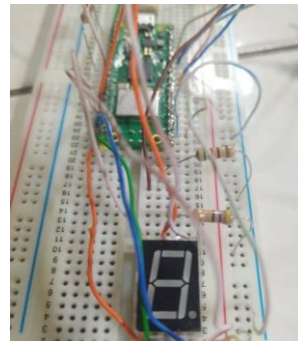
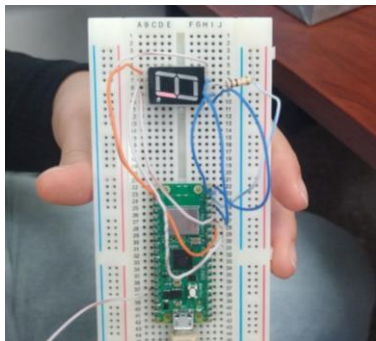
IV. Resultados

A lo largo del desarrollo del proyecto se presentaron muchos desafíos, pues era la primera vez que se trabajaba aplicando tanto los conocimientos de programación como de electrónica aprendidos hasta el momento. Fue una experiencia muy retadora, pero al mismo tiempo muy enriquecedora, ya que permitió la mejora y fortalecimiento de habilidades y conocimientos aprendidos previamente. Seguidamente, se hará un desglose de los resultados obtenidos.

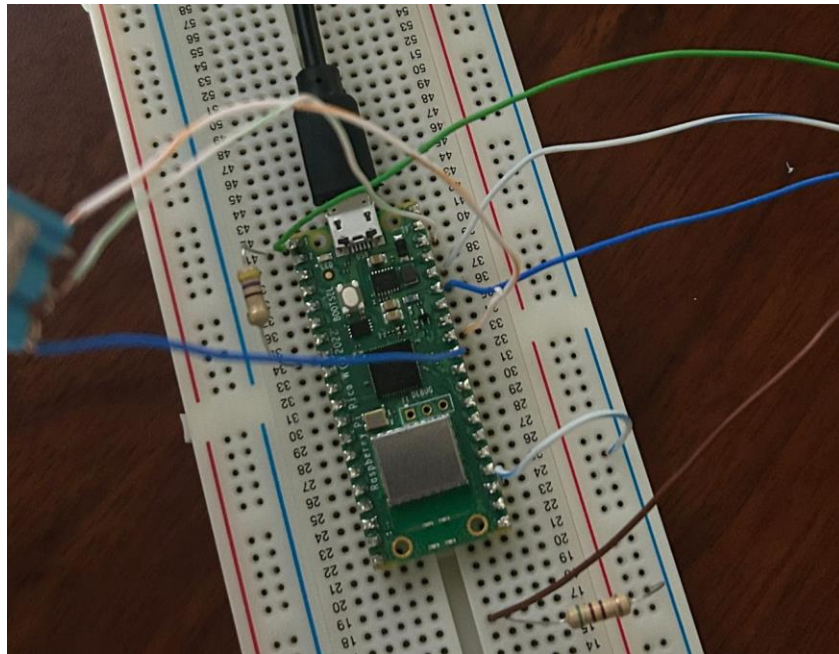
Para dar inicio, el componente que recibió más atención fueron los botones, estos eran un poco complicados de conectar, así que se requirió la ayuda de varias personas para conseguir su funcionamiento. Estos traían 5 patas: 2 para el *led*, 2 que controlan la lógica del botón (NO o NC) y el común que transmite el voltaje. Al final, por el tipo de proyecto, se conectó la pata que hacía que el botón fuera NO (*Normally Open*).



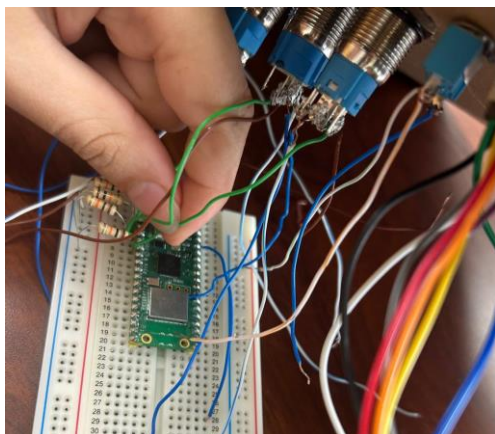
Seguidamente, el 7 segmento fue el segundo que más tiempo requirió, pues las primeras veces que se intentó montar en el circuito no funcionaba. Al final, se decidió pedirle ayuda al profesor y se corrigieron los detalles que no permitían su funcionamiento. Se conectaron todos los segmentos a excepción del punto, y se decidió que la pata común iría a tierra. Para su conexión final se usaron cables macho – hembra, pues soldarlo resultaba muy complicado por la cercanía de las patas entre sí.



Ahora bien, en cuanto al *buzzer* y el *switch*, su conexión fue relativamente sencilla y se logró al primer intento. El *switch* es de tres modos, lo cual fue realmente útil a la hora de programar los niveles del juego según su dirección. En cuanto al *buzzer*, este es pasivo, aunque cabe destacar que esto no dificultó en nada durante el desarrollo del código o conexión. Sin embargo, algo que sí fue complicado acerca de ambos componentes es que su capa exterior era de plástico y las patas quedaban considerablemente cerca de ella, lo cual hizo imposible llevar a cabo la soldadura. Al final se tuvo que optar por amarrar el cable a cada pata para poder trabajar con ellos en la maqueta final.



Finalmente, en cuanto al montaje en la maqueta final, este fue un poco tedioso, pues por la cantidad de cables de los componentes era difícil mantener el orden. Aquí, nuevamente, los botones presentaron ciertas complicaciones, ya que no se lograba que estos quedaran lo suficientemente firmes como para ser presionados y no caerse. Al final tuvo que utilizarse otra caja y cambiar el tipo de hueco que se estaba haciendo para colocarlos, esto fue de gran ayuda y permitió que los botones quedaran firmes. Los otros tres componentes no presentaron este tipo de dificultad ya que eran más ligeros y requerían un menor contacto.



Al finalizar el proyecto se obtuvo una máquina funcional y que cumple con cada uno de los atributos pedidos. La máquina final cuenta con tres niveles de dificultad, los cuales pueden ser escogidos según la dirección del *switch*. Asimismo, se le implementó un modo especial, este es el Modo Piano, el cual permite al jugador crear melodías con los botones, este modo se hizo inspirado en el juego llamado *Piano Tiles*. Cabe destacar, que no solo el Modo Piano viene acompañado de sonido, sino que cada nivel cuenta con sonidos especiales para motivar al usuario. Además, se cuenta con un 7 segmento que permite mostrar los puntos obtenidos al jugador con cada acierto que obtenga.

Ahora bien, en cuanto a la maqueta donde se acomodaron y distribuyeron los elementos, esta se realizó con una caja de cartón, en la cual en la parte de adentro se encuentra todo el circuito montado en una *protoboard* junto a la *Raspberry Pi Pico W*. En la parte exterior se encuentran los 4 botones acomodados en línea horizontal en la parte baja de la caja, en el centro se encuentra el 7 segmento, en el costado superior izquierdo el *buzzer* y en el derecho el *switch*.

Continuando lo anterior, en cuanto a la estética de la maqueta esta es bastante simple, pues es color cartón y no cuenta con ningún elemento o detalle adicional. En un inicio se planeaba hacerla con temática de *Star Wars*, sin embargo, a medida que se fue avanzando con el proyecto se presentaron algunas dificultades, las cuales requirieron más tiempo del imaginado para poder ser resueltas. Esto llevó a que la estética de la maqueta pasara a segundo plano, pues la parte más importante a llevar a cabo era la funcionalidad del sistema.

En resumen, a pesar de los desafíos encontrados, especialmente en la distribución de tiempo y recursos, se priorizó la funcionalidad del sistema sobre la estética, asegurando que cada componente cumpliera su propósito dentro de la maqueta. Este proceso permitió un aprendizaje significativo en la programación con *MicroPython* y el manejo de hardware, consolidando conocimientos clave para futuros proyectos.

V. Conclusiones

Los resultados sugieren que unos botones más simples hubiesen sido de gran ayuda a la hora de la elaboración del proyecto, ya que una de las principales dificultades siempre fue el funcionamiento de ellos. Aquí mismo cabe destacar que, una maqueta con otro color, como negro o gris metálico, hubiese ayudado a resaltar el *led* de cada botón, ayudando esto también a que el usuario mejorara su tiempo de reacción al ver la luz de una forma más clara. En general, se resalta la importancia del uso de componentes mecánicos más simples para facilitar el desarrollo del sistema, así como de la creación de interfaces que hagan contraste con los indicadores luminosos para mejorar la experiencia del usuario.

Por otra parte, el desarrollo del proyecto no solo permitió mejorar en el trabajo en equipo y administración del tiempo, sino que también impulsó un crecimiento significativo en habilidades técnicas y resolución de problemas. Durante cada jornada de trabajo, se pudo observar una mejora progresiva en aspectos clave como la soldadura, el montaje de circuitos y la identificación de fallos, lo que permitió afianzar conocimientos esenciales en electrónica y programación. Además, la experiencia práctica fortaleció la capacidad de adaptación ante imprevistos, promoviendo una mentalidad analítica y estructurada para la solución de problemas en sistemas complejos. Esta evolución en habilidades y conocimientos no solo resultó beneficiosa para la ejecución del proyecto actual, sino que también preparó el camino para el desarrollo de futuros sistemas, donde la experiencia adquirida servirá como base para enfrentar desafíos más avanzados con mayor eficiencia y confianza.

En cuanto a la parte del código, debido al conocimiento previo que se tiene sobre Python, el proceso de desarrollar el código no fue tan complicado, ya que la lógica de programación no resultó tan diferente a la que ya se conocía, el proceso de depuración constante fue clave para lograr un código funcional y optimizado, demostrando que la iteración y corrección de errores son fundamentales en el desarrollo de sistemas embebidos. Se implementó una lógica eficiente haciendo uso de *MicroPython*, lo cual permitió trabajar de forma directa y flexible con el hardware. Gracias a esto, se logró la creación de distintos modos de juego, un modo especial (Modo Piano), el conteo del puntaje y la incorporación de animaciones y sonidos, contribuyendo así a la fluidez del sistema. El uso de *MicroPython* permitió y facilitó la interacción con los pines, el uso de temporizadores, sonidos y en general el control preciso de los componentes electrónicos.

VI. Referencias bibliográficas

6 - *Introducción a la Protoboard y sus Componentes | Curso Arduino (Español)*. (s. f.).

<https://cursoarduino.powarsteam.com/introduccion/espanol/indice/6-introduccion-a-la-protoboard-y-sus-componentes>

Garrett, C. (2023, 25 enero). *MicroPython: An Intro to Programming Hardware in Python*.

<https://realpython.com/micropython/>

Sibaja, R. Z. (2024, 16 marzo). *Explorando los sistemas embebidos: inteligencia en cada*

dispositivo. U Fidéлитas. <https://ufidelitas.ac.cr/que-son-los-sistemas-embebidos/>